To-Do App

Feature Spec - Submittable Version

Expo (React Native) | Express + TypeScript + MongoDB | JWT Auth | Zustand

Scoped directly to the assignment's requirements, with the microservices version kept as an optional follow-up.

1. Scope & Approach

This version is scoped tightly to what the assignment actually asks for and grades on: a working Expo app with registration, login, and full task CRUD, backed by one clean Express + MongoDB API. The distributed/microservices architecture explored earlier is parked as an optional stretch goal, attempted only after this version is fully working end-to-end.

Confirmed stack

- Mobile app: Expo (React Native), TypeScript, tested via Expo Go on a physical Android phone
- Final Android build: EAS Build (eas build -p android) or expo prebuild, to produce an installable APK
- Authentication: custom JWT (access + refresh token), bcrypt password hashing
- Backend: single Express + TypeScript API, cleanly layered (routes / controllers / models / middleware)
- Database: MongoDB with Mongoose
- State management: Zustand

---

2. Backend - Project Structure

One codebase, clearly separated by responsibility - not split into deployable services, but organized so a reviewer can tell auth logic from task logic at a glance.

- src/config - database connection, environment loading
- src/models - User.ts, Task.ts (Mongoose schemas)
- src/middleware - authMiddleware.ts (verifies JWT, attaches userId to request), errorHandler.ts
- src/controllers - authController.ts, taskController.ts (business logic)
- src/routes - authRoutes.ts, taskRoutes.ts (thin route definitions)
- src/utils - token generation/verification helpers, validation helpers
- src/server.ts - app entrypoint, wires everything together

2.1 API endpoints

Auth

- POST /api/auth/register - email + password, validates and hashes, creates user
- POST /api/auth/login - verifies credentials, returns access + refresh token
- POST /api/auth/refresh - exchanges refresh token for a new access token
- GET /api/auth/me - returns the logged-in user's profile (protected route)

Tasks (all routes protected - require a valid access token)

- POST /api/tasks - create a task (title, description, dueDateTime, priority)
- GET /api/tasks - list the user's tasks; supports ?status=, ?priority=, ?sort=
- GET /api/tasks/:id - get one task (must belong to the requesting user)
- PATCH /api/tasks/:id - update any field, including marking complete
- DELETE /api/tasks/:id - delete a task

---

3. Data Models

User

- _id, email (unique, required), passwordHash, createdAt

Task

- _id, userId (ref to User, always filtered on in queries)
- title (required), description
- dueDateTime, deadline, priority: 'low' | 'medium' | 'high'
- status: 'pending' | 'completed'
- tags (bonus), createdAt, updatedAt

---

4. Mobile App - Screens & Features

4.1 Auth flow

- Splash/bootstrap: check SecureStore for a saved token, auto-login if valid
- Register screen: email, password, confirm password, inline validation
- Login screen: email, password, error state for invalid credentials
- Logout: clears token from Zustand + SecureStore

4.2 Task screens

- Task list: shows title, priority badge, due date, checkbox to mark complete, filter chips (All/Pending/Completed)
- Add task: title, description, deadline picker, priority selector
- Edit task: same form, pre-filled, updates existing task
- Delete: swipe or button, with confirmation
- Empty state and pull-to-refresh

4.3 State management (Zustand)

- authStore: token, user, isAuthenticated, login/register/logout actions
- taskStore: tasks[], filters, loading state, CRUD actions calling the API layer

---

5. UI Design

Mockups for the three core screens - login, task list, and add task - were reviewed in chat alongside this document (login card, task list with priority-color-coded cards and filter chips, and the add-task form with a priority selector). Key visual decisions:

- Priority color-coding: high = red/coral, medium = amber, low = green/teal - applied consistently as small badges on every task card
- Filter chips (All / Pending / Completed) at the top of the task list for quick switching
- Completed tasks shown with a checked box, strikethrough title, and reduced opacity rather than being hidden

- Single accent color used sparingly (primary buttons, active filter chip) so the priority colors stay the strongest visual signal on screen
- Empty states and a floating/inline 'New task' button so the primary action is always reachable
- Consistent spacing and card-based layout (rounded corners, hairline borders, no heavy shadows) for a clean, modern feel

These are direction-setting mockups, not pixel-perfect specs - treat spacing, exact colors, and copy as adjustable during implementation.

---

6. Build Order

- 1. Express + MongoDB backend: User + Task models, auth routes, task routes, tested with Postman/curl
- 2. Expo app scaffold: navigation (auth stack + app stack), Zustand stores, API client
- 3. Wire up Register/Login screens to the live backend
- 4. Task list screen fetching real data, then Add/Edit/Delete/Complete
- 5. Apply the UI design: priority colors, filter chips, empty states
- 6. Polish: loading states, error handling, pull-to-refresh
- 7. Build the Android APK via EAS Build and do a final device test
- 8. Stretch (only after everything above works): smart priority+deadline sort, tags, dark mode, offline sync, or the microservices split explored earlier

---

7. Mapping to the Assignment's Evaluation Criteria

Evaluation criterion	Where it's covered
Correctness	Section 2.1 + 4.1-4.2 - full register/login/CRUD flow
Code quality	Section 2 - layered structure, comments per the assignment's ask
User Interface	Section 5 - color-coded priorities, filters, clean card layout
State management	Section 4.3 - Zustand stores for auth and tasks
Auth flow	Section 2.1 (Auth) - JWT issue/refresh, protected routes
Bonus features	Section 6, step 8 - sort algorithm, tags, dark mode, offline sync